



Department of Computer Science & Engineering

Course Title: Machine Learning

Course Code: UE24CS352A

Semester: V

Academic Session: Aug–Dec 2026

Machine Learning Laboratory 3

Exploratory Decision Trees

1. Introduction

A decision tree classifier makes predictions by repeatedly splitting the data on the feature and threshold that best separates the classes at each step, until it reaches a stopping condition. It is one of the few classical models whose decision process can be read directly — every prediction traces back to a short sequence of “if feature $\leq$ value” checks.

This lab is not about learning the `.fit()` / `.predict()` API — that ground is already covered. It is about building intuition for how a decision tree actually behaves: how the choice of impurity criterion (Gini vs. Entropy) affects which splits get chosen, how tree depth trades off underfitting against overfitting, how post-hoc pruning differs from early stopping, and why a single train/test split can mislead you about which model is genuinely best. Work is done on the Breast Cancer Wisconsin (Diagnostic) dataset, a small, clean, binary-classification dataset built into scikit-learn, so the focus stays on model behaviour rather than data cleaning.

2. Lab Objectives

After completing this laboratory exercise, students should be able to

- explain how the Gini and Entropy impurity measures differ, and when — if ever — they lead to different trees
- demonstrate, with a plot, how training and test accuracy diverge as tree depth increases, and identify the depth at which overfitting begins
- apply cost-complexity pruning (`ccp_alpha`) as an alternative to early stopping, and explain how the two mechanisms differ
- use `GridSearchCV` with cross-validation to search a hyperparameter space, and interpret its output correctly

- read a confusion matrix to identify which error type a model is making, rather than relying on a single accuracy number
- connect a model's feature importances back to the structure of its decision tree
- critically evaluate whether one model “beating” another on a single test set is meaningful evidence that it generalizes better

3. Course Outcomes

CO	Learning Outcome
CO1	Explain how split-selection criteria (Gini, Entropy) shape a decision tree's structure.
CO2	Apply hyperparameter control techniques — max_depth, cost-complexity pruning, and grid search — to manage overfitting.
CO3	Evaluate a classification model using metrics and diagnostics beyond a single accuracy figure.
CO4	Interpret model outputs (confusion matrices, feature importances, tree diagrams) to explain model behaviour.
CO5	Compare model-selection strategies (single train/test split vs. cross-validation) and justify which is more trustworthy.

4. Software Requirements

- Google Colab / Jupyter Notebook
- Python 3
- NumPy, Pandas
- Matplotlib, Seaborn
- scikit-learn — datasets, model_selection, tree, metrics

5. Dataset

Dataset Name: Breast Cancer Wisconsin (Diagnostic)

Loaded directly from `sklearn.datasets.load_breast_cancer()` — no download, no file handling, and no missing values to clean up. It ships as part of scikit-learn precisely because it is small, clean, and well-separated, which keeps the lab's attention on tree behaviour rather than data wrangling.

Each of the 569 samples is one digitized image of a fine-needle-aspirate (FNA) biopsy of a breast mass. For each image, ten base measurements are computed from the cell nuclei visible in it:

- radius — mean distance from the nucleus centre to its perimeter
- texture — standard deviation of grayscale intensity values
- perimeter and area — size of the nucleus boundary

- smoothness — local variation in radius length
- compactness — $\text{perimeter}^2 / \text{area} - 1$, penalizes irregular shapes
- concavity — severity of concave (inward-curving) portions of the boundary
- concave points — number of concave portions of the boundary
- symmetry — how symmetric the nucleus shape is
- fractal dimension — a “coastline approximation” measure of boundary complexity

Every image contains many nuclei, not one, so each of these ten measurements is summarized three ways across all nuclei in the image: its mean, its standard error, and its “worst” value (the mean of the three largest measurements seen). 10 measurements $\times$ 3 summary statistics gives the dataset’s 30 numeric features — for example mean radius, radius error, and worst radius are three separate columns, all derived from the same underlying measurement. All 30 features are continuous and already on comparable numeric scales, which is part of why this dataset is a clean starting point: no encoding of categorical variables and no unit mismatches to reason about.

The target is binary: malignant (0) or benign (1).

The class split is 357 benign (62.7%) vs. 212 malignant (37.3%) — mildly imbalanced. A classifier that always predicted “benign” would already score roughly 63% accuracy, so raw accuracy numbers throughout the lab should be read against that floor, not against 50%. This becomes directly relevant again in Section 8.

6. Working Through the Notebook

The notebook is organized into 11 sections. Sections 1–3 are fully worked — run them as-is; they set up the data, split, and baseline model that every later section depends on. From Section 4 onward, cells marked TODO contain instructions but no solution code. Several TODOs specify an exact variable name to use — later cells depend on that exact name, so work through the sections in order rather than skipping around.

Section 1 — Setup & Dataset

Imports every library used in the notebook, fixes a random seed for reproducibility, and loads the dataset into a feature DataFrame X and target Series y. Fully worked — just run it, but confirm the printed shape (569 rows $\times$ 30 columns) matches what Section 5 of this manual describes before moving on.

Section 2 — Quick EDA

Three quick visual checks before any modelling begins: a class-balance bar chart (confirming the 63/37 split described above), a correlation heatmap of the first 10 features (several of the size-related features — radius, perimeter, area — are highly correlated with each other, which is expected since they all describe the same physical property), and KDE plots comparing how two individual features separate across the malignant/benign classes. Fully worked, apart from one reflection question on class balance.

Section 3 — Baseline Model

Creates a train/test split (80% train, 20% test, stratified so both splits keep the same 63/37 class ratio), then fits a plain, default-parameter `DecisionTreeClassifier` as a reference point. Also defines a reusable `plot_confusion()` helper function that every later section calls to visualize errors. Every model built for the rest of the lab is measured against this baseline's accuracy. Fully worked.

Section 4 — Gini vs. Entropy

Concept

A decision tree decides where to split by measuring impurity — how mixed the classes are in a node — and picking the split that reduces impurity the most. Gini impurity and entropy are the two standard ways to measure that. Gini impurity is the criterion historically used by CART (Classification and Regression Trees), the algorithm scikit-learn's `DecisionTreeClassifier` is built on; entropy (measured in bits, via information gain) is the criterion historically used by ID3 and its successor C4.5, an earlier and different family of tree-building algorithms. scikit-learn does not implement ID3 or C4.5 directly — it is a CART-style implementation throughout — but it lets you swap in entropy as the impurity measure via the criterion parameter, which is exactly what this section explores. In practice the two measures behave very similarly and often select the same or near-identical splits.

Train one tree per impurity criterion, holding every other parameter fixed, and compare their resulting accuracy and depth. First section requiring you to write code.

Section 5 — Depth Exploration

Sweep `max_depth` from 1 to 20, recording train and test accuracy separately at each depth. This produces the single most important plot in the lab — the classic overfitting curve where train accuracy keeps climbing toward 100% while test accuracy peaks at some moderate depth and then plateaus or declines. The gap that opens up between the two lines as depth increases is overfitting made visible.

Section 6 — Accuracy vs. Depth, split by Criterion

Repeats the depth sweep from Section 5, but separately for each criterion, to check whether the optimal depth shifts depending on which impurity measure is used. The output of this section becomes the “manual best” model that Section 9's grid search is later compared against.

Section 7 — Pruning

Concept

`max_depth` controls overfitting by stopping the tree early — it is never allowed to grow past a certain level in the first place. Cost-complexity pruning takes the opposite approach: let the tree grow all the way out, then trim branches back afterward based on how little accuracy each branch actually adds relative to the complexity it costs. The parameter **`ccp_alpha`** controls how aggressively this trimming happens — at $\alpha = 0$, nothing is pruned (the full tree is kept); as α increases, larger and larger branches get cut away, until at a high enough α the tree is trimmed all the way down to a single leaf. scikit-learn's **`cost_complexity_pruning_path()`** function computes the sequence of α values at which a branch would next get pruned, which is what this section sweeps over.

The same train/test-vs-complexity curve as Section 5 is plotted here, but against alpha instead of depth, so the two mechanisms' effects on overfitting can be compared directly.

Section 8 — Confusion Matrix Deep-Dive

Build a shallow, a near-optimal, and an overly deep tree, and compare their confusion matrices side by side. Accuracy alone will look similar across the three — the point of this section is to show that the error composition underneath is not: which specific mistakes each model makes (missed cancers vs. false alarms) can differ substantially even when the headline accuracy numbers look close.

Section 9 — Grid Search

Concept

Sections 5 and 6 searched for a good configuration manually — sweep a range of values, look at a plot, pick whichever scored highest on the one test set. GridSearchCV automates and strengthens that process in two ways. First, it exhaustively tries every combination of hyperparameter values you give it (a “grid” of options) rather than relying on you to sweep them by hand. Second, and more importantly, it evaluates every combination using k-fold cross-validation instead of a single train/test score: the training data is split into k folds (5, in this lab), and each candidate configuration is trained and scored k times, each time holding out a different fold as validation. The average score across those k folds — not a score from any single split — is what decides the winner. This matters because a single train/test split can make one configuration look better than it really is purely by chance; averaging over several folds is far less sensitive to that kind of luck when choosing which configuration to trust.

Use GridSearchCV with 5-fold cross-validation to search jointly over criterion and max_depth. Its selected model is compared against the manual best from Section 6, prompting reflection on why cross-validated selection and a single train/test score can disagree — and why a higher test-set score does not automatically mean a genuinely better model.

Section 10 — Bonus: Feature Importance & Tree Visualization

Plot the top-10 most important features for the grid-search model, and visualize the top levels of the tree itself, connecting what the tree actually splits on back to the importance ranking. Feature importance in a decision tree comes directly from how much impurity each feature's splits reduce, weighted by how many samples pass through that split — so the feature chosen at the root, which sees every training sample, tends to dominate the importance ranking.

Section 11 — Reflection / Write-up

Four short-answer questions synthesizing the whole lab: which hyperparameter mattered most for overfitting, whether Gini and Entropy ever meaningfully disagreed, why accuracy alone can be misleading, and how max_depth, ccp_alpha, and grid search each control complexity differently.

7. Deliverable

Submit the completed notebook (.ipynb), with:

- every TODO code cell implemented and producing output
- every reflection question answered in your own words (2–4 sentences unless stated otherwise)
- all cells run in order, top to bottom, with outputs visible — a notebook that only works when cells are run out of order will not be accepted